

Machine Learning Project: Fuzzy Clustering

Cristian Vázquez

February 4, 2025

Contents

1	Overview of Clustering Methods	2
2	Hard vs Fuzzy Partitions	4
2.1	Hard Partitions	4
2.2	Fuzzy Partitions	5
3	Fuzzy c-Means Functional	6
4	Basic Fuzzy c-Means Algorithm (FCM)	6
5	Parameter Selection for the FCM Algorithm	9
5.1	Number of Clusters (c)	9
5.2	Fuzziness Parameter (m)	10
5.3	Norm-Inducing Matrix ($\mathbf{A}$, $\ \cdot\ _A$)	10
5.4	Initial Partition of $\mathbf{W}$	11
6	Limitations of the Basic FCM Algorithm	11
7	Extensions of the Basic FCM Algorithm	12
7.1	Adaptative distance measure algorithms	12
7.1.1	Gustafson-Kessel Algorithm	13
7.2	Application of the GK algorithm to synthetic data	15
7.3	Possibilistic c -Means Algorithm	16
7.4	Other Extensions of the Basic FCM Algorithm	18
8	Application to Data	19
8.1	Image Segmentation with Fuzzy Clustering	19
9	Conclusions	21
A	Python codes	22

1 Overview of Clustering Methods

Clustering is a fundamental task in unsupervised machine learning which aims to partition a dataset into meaningful groups, or clusters, based on similarity measures. Numerous clustering algorithms have been proposed in the literature, which can be broadly classified based on the nature of the subsets they form (either hard or fuzzy) and the algorithmic approach they employ.

Hard clustering methods are based on classical set theory. In these methods, each object either fully belongs to a cluster or does not belong to it at all. This approach partitions the data into mutually exclusive subsets, meaning an object cannot belong to more than one cluster. Fuzzy clustering methods allow objects to belong to multiple clusters simultaneously, with varying degrees of membership. This flexibility makes fuzzy clustering more natural for many applications, particularly when objects lie near the boundaries of clusters. Instead of assigning a strict membership, fuzzy clustering provides a degree of belonging between 0 and 1 for each cluster. This approach also overcomes the analytical challenges of hard clustering since the membership values are differentiable and can be incorporated into optimisation algorithms.

Clustering algorithms can also be categorised by their computational approach:

- **Hierarchical Methods:** These include agglomerative and divisive methods, which iteratively form or split clusters by reallocating memberships one point at a time based on some suitable measure of similarity (single, complete or average linkage).
- **Graph-Theoretic Methods:** Here, the dataset is represented as a graph where nodes correspond to data points and edge weights signify the similarity between points. Clustering is then performed using graph-theoretic concepts (for instance, spectral clustering).
- **Objective Function-Based Methods:** These methods optimise a defined objective function (distortion) to identify desirable partitions. Nonlinear optimisation techniques are employed to search for local optima of this objective function.

Non-hierarchical clustering methods, such as the *K-means* algorithm, are among the most widely used techniques due to their simplicity and efficiency. We provide a quick summary of this method, following the standard notation of [4]. The objective of *K-means* clustering is to partition a set of N observations, $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, in a D -dimensional Euclidean space into K clusters, where the number of clusters K is a prescribed parameter. Each cluster $\mathcal{C}_k$ is represented by a prototype vector or centroid $\boldsymbol{\mu}_k$, $k = 1, \dots, K$, which corresponds to the centre of mass of that cluster. The *K-means* algorithm iteratively assigns each data point $\mathbf{x}_n$ to the nearest cluster centre, $\boldsymbol{\mu}_k$, and updates the cluster centres to minimise the total distortion measure J , defined as:

$$J = \sum_{n=1}^N \sum_{k=1}^K w_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2, \quad (1)$$

where w_{nk} is a binary indicator variable that equals 1 if data point $\mathbf{x}_n$ is assigned to cluster k , and 0 otherwise and $\|\cdot\|$ denotes the Euclidean norm. The *K-means* algorithm alternates then between two steps:

1. **Assignment Step:** For each data point $\mathbf{x}_n$, assign it to the cluster k whose centroid $\boldsymbol{\mu}_k$ minimises the distance:

$$w_{nk} = \begin{cases} 1 & \text{if } k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2, \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

2. **Update Step:** Recalculate the cluster centres as the mean of all data points assigned to that cluster in the previous step:

$$\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N w_{nk} \mathbf{x}_n}{\sum_{n=1}^N w_{nk}} \in \mathbb{R}^D, \quad k = 1, \dots, K. \quad (3)$$

These steps are repeated until convergence, typically when the cluster assignments no longer change or the distortion measure J reaches a minimum. Convergence of this algorithm is guaranteed, but may be to a local (and not global) minimum of the distortion. One notable feature of the *K-means* algorithm is that at each iteration, every data point is assigned to a single cluster. While some data points will be much closer to a particular centre $\boldsymbol{\mu}_k$ than to any other centre, there may be other data points that lie roughly midway between cluster centres. In that case case, it is not clear that the hard assignment to the marginally closer cluster is the most appropriate choice.

While *K-means* is effective in many scenarios, it has significant limitations that motivate the development of more flexible approaches like fuzzy clustering: hard assignments fail to reflect inherent uncertainty in overlapping regions, they struggle with data ambiguity since it forces strict assignments to one cluster, outliers and noise can disproportionately affect cluster centroids and strict rigid cluster boundaries may not align with natural data structures.

Fuzzy clustering extends traditional hard clustering by allowing data points to belong to multiple clusters with varying degrees of membership, represented by values between 0 and 1. This soft assignment approach introduces a probabilistic formulation that effectively captures the uncertainty in cluster assignments, making it particularly robust in scenarios involving overlapping clusters and noisy data. By incorporating membership values w_{nk} , fuzzy clustering offers a more flexible and nuanced representation of the underlying data structure, addressing key limitations of hard-clustering methods like *K-means*. This report focuses on fuzzy clustering methods that optimise an objective function. These methods have been extensively studied [1, 3], with well-established mathematical properties related to convergence and cluster validity.

2 Hard vs Fuzzy Partitions

Clustering techniques aim to divide a dataset into groups, or clusters, based on specific similarity criteria. To extend the rigid assignments of hard clustering methods and establish a robust theoretical framework, it is essential to first define a hard partition mathematically in terms of classical subsets. This foundational concept can then be generalised to accommodate soft clustering techniques. The advent of fuzzy logic and fuzzy sets provided the theoretical basis that facilitated the development of fuzzy clustering methods. In this context, we begin by introducing the two fundamental types of partitions used in clustering: **hard partitions** and **fuzzy partitions**.

2.1 Hard Partitions

Hard partitions can be formulated using classical set theory, where each data point strictly belongs to exactly one cluster [3]. Formally, given a dataset of observations $\{\mathbf{x}_k\}_{k=1}^N \subset \mathbb{R}^D$ represented as a $D \times N$ matrix $\mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_N]$, a hard partition of $\mathbf{X}$ in $c \in \mathbb{N}$ clusters¹, is defined as a family of subsets $\{A_i : 1 \leq i \leq c\} \subset \mathcal{P}(\mathbf{X})$ that satisfy the following properties:

1. $\bigcup_{i=1}^c A_i = \mathbf{X}$, ensuring that all data points are included in some cluster.
2. $A_i \cap A_j = \emptyset$, for $i \neq j$, ensuring that the clusters are mutually exclusive (pairwise disjoint).
3. $\emptyset \subsetneq A_i \subsetneq \mathbf{X}$, meaning no cluster is empty or contains the entire dataset.

These conditions are easily generalised if we recast this definition in matrix form, introducing the membership functions. Then a hard partition can be represented by a membership matrix $\mathbf{W} = [w_{ik}]_{c \times N}$. Here, $w_{ik} := w_i(k)$ is defined as a piecewise function:

$$w_{ik} = \begin{cases} 1, & \text{if } \mathbf{x}_k, \text{ the } k\text{-th data point, belongs to } A_i, \text{ the } i\text{-th cluster,} \\ 0, & \text{otherwise.} \end{cases}$$

Thus, the i -th row of the matrix $\mathbf{W}$ contains the values of the membership function of A_i , the i -th subset of $\mathbf{X}$, so that $\mathbf{W}$ is usually termed a hard c -partition of $\mathbf{X}$. The conditions for a hard partition matrix are:

1. $w_{ik} \in \{0, 1\}$, $1 \leq i \leq c$, $1 \leq k \leq N$, ensuring binary membership values.
2. $\sum_{i=1}^c w_{ik} = 1$, $1 \leq k \leq N$, ensuring that each data point belongs to exactly one cluster (sum by columns of the matrix).
3. $0 < \sum_{k=1}^N w_{ik} < N$, $1 \leq i \leq c$, ensuring that no cluster is empty or contains all data points.

¹As contrast to K -means clustering method, it is standard notation in the literature to work with c clusters when working with fuzzy partitions (the first fuzzy clustering method developed was precisely the fuzzy c -means).

We define the space of all possible hard partition matrices for $\mathbf{X}$ as

$$M_{hc} = \left\{ \mathbf{W} \in \mathbb{R}^{c \times N} \left| w_{ik} \in \{0, 1\}, \forall i, k; \sum_{i=1}^c w_{ik} = 1, \forall k; 0 < \sum_{k=1}^N w_{ik} < N, \forall i \right. \right\}.$$

Hard partitions are computationally simpler and easier to implement, but often fail to represent realistic data accurately: in datasets where clusters overlap or contain boundary points, hard clustering enforces an artificial exclusivity, which may not capture the intricacies of the data structure. As we have already mentioned, these shortcomings can be overcome using fuzzy partitions.

2.2 Fuzzy Partitions

Fuzzy partitions generalise hard partitions by allowing data points to belong to multiple clusters simultaneously, with varying degrees of membership. A fuzzy partition can be again represented by a membership matrix $\mathbf{W} = [w_{ik}]_{c \times N}$, with elements now in the unit interval $[0, 1]$. Here, w_{ik} represents the degree of membership of the k -th data point in the i -th cluster. Unlike hard partitions, in fuzzy partitions the values that a membership function w_i in a fuzzy subset can take range from 0 to 1. The conditions for a fuzzy partition matrix are:

1. $w_{ik} \in [0, 1]$, $i = 1 \dots, c$, $k = 1, \dots, N$, ensuring valid membership values.
2. $\sum_{i=1}^c w_{ik} = 1$, $k = 1 \dots, N$, ensuring that the total membership for each data point across all clusters equals 1 (this column normalisation actually ensures that the membership is a probability measure so that for each data point $\mathbf{x}_k$, $w_k : \{1, \dots, c\} \rightarrow [0, 1]$ is indeed a probability function, so that the total membership of the given point equals one).
3. $0 < \sum_{k=1}^N w_{ik} < N$, $i = 1 \dots, c$, ensuring that clusters are non-empty and do not cover all data points entirely.

As observed, the primary distinction between fuzzy and hard partition matrices lies in the first property. Notably, hard partitions can be regarded as a special case of fuzzy partitions, meaning that fuzzy clustering algorithms are capable of generating hard c -partitions as well. We define the space of all possible fuzzy partition matrices for $\mathbf{X}$ as

$$M_{fc} = \left\{ \mathbf{W} \in \mathbb{R}^{c \times N} \left| w_{ik} \in [0, 1], \forall i, k; \sum_{i=1}^c w_{ik} = 1, \forall k; 0 < \sum_{k=1}^N w_{ik} < N, \forall i \right. \right\}.$$

This definition extends and enriches the traditional concept of partitions, providing greater flexibility and expressiveness ($M_{hc} \subset M_{fc}$). Fuzzy clustering methods, therefore, aim to identify the optimal fuzzy c -partition of the dataset $\mathbf{X}$. Of the different methods the most popular and studied method is the one associated with the generalised least-square errors functional (distortion) introduced in the following section.

3 Fuzzy c -Means Functional

The fuzzy c -means functional is a widely used foundation for many analytical fuzzy clustering algorithms. It is formulated as an optimisation problem where the aim is to minimise a specific cost function called distortion, which balances membership degrees and cluster compactness. This functional and subsequent algorithm was first proposed by J.C Dunn (1974) [5] and later improved by Bezdek (1981) [3].

Given a dataset of D -dimensional observations $\{\mathbf{x}_k\}_{k=1}^N \subset \mathbb{R}^D$ represented as a $D \times N$ matrix $\mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_N] \in \mathbb{R}^{D \times N}$, and a predefined number $c \in \mathbb{N}$, the fuzzy c -means functional is defined as:

$$J(\mathbf{X}; \mathbf{W}, \mathbf{M}) = \sum_{i=1}^c \sum_{k=1}^N (w_{ik})^m \|\mathbf{x}_k - \boldsymbol{\mu}_i\|_A^2, \quad (4)$$

where:

- $\mathbf{W} = [w_{ik}] \in \mathbb{R}^{D \times N}$ is the fuzzy partition matrix ($\mathbf{W} \in M_{fc}$), where $w_{ik} \in [0, 1]$ denotes the membership degree of data point $\mathbf{x}_k$ in cluster i , and this matrix satisfies the defining properties stated in 2.2.
- $\mathbf{M} = \{\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \dots, \boldsymbol{\mu}_c\}$ is the set of cluster prototypes (centroids), with $\boldsymbol{\mu}_i \in \mathbb{R}^D$, $i = 1 \dots, c$.
- $\|\mathbf{x}_k - \boldsymbol{\mu}_i\|_A^2$ is the squared distance between $\mathbf{x}_k$ and $\boldsymbol{\mu}_i$, weighted by some positive definite matrix $\mathbf{A} \in \mathbb{R}^{D \times D}$. The induced distance given by this selected matrix is $\|\mathbf{x}_k - \boldsymbol{\mu}_i\|_A^2 = d_A(\mathbf{x}_k, \boldsymbol{\mu}_i)^2 := D_{ik\mathbf{A}}^2 = (\mathbf{x}_k - \boldsymbol{\mu}_i)^\top \mathbf{A} (\mathbf{x}_k - \boldsymbol{\mu}_i)$, $1 \leq i \leq c$, $1 \leq k \leq N$.
- $m \in [1, \infty)$ is the fuzziness exponent or fuzzifier, a parameter controlling the degree of fuzziness of the desired partition.

The value of the cost function $J(\mathbf{X}; \mathbf{W}, \mathbf{M})$ measures the total variance of data points from their respective cluster centroids (squared error), weighted by their membership degrees. Optimal fuzzy clusterings of $\mathbf{X}$ are just pairs of fuzzy partitions and corresponding centroids, $(\mathbf{W}, \mathbf{M})$, that locally minimise the distortion functional $J(\mathbf{X}; \mathbf{W}, \mathbf{M})$. When $m = 1$, this is exactly the distortion function used in k -means (eq. 1), but with c clusters and with fuzzy partition matrices that obviously include the hard partition matrices of k -means. The most popular fuzzy clustering algorithm consists of finding the optimal fuzzy partition of the data set that minimises this least-squared error functional, which led to the development of the basic fuzzy c -means algorithm, which we explain in detail in the following section. This basic algorithm which was also introduced in [5] and developed by [3].

4 Basic Fuzzy c -Means Algorithm (FCM)

The fuzzy c -means (FCM) algorithm is one of the most widely used methods for fuzzy partitioning, serving as the foundation for many advanced fuzzy clustering techniques. It

can be viewed as a straightforward generalisation of the classical k -means algorithm for hard clustering (connections to Equations (2) and (3) will be made clear later). The minimisation of the c -means functional represents a nonlinear optimisation problem that can be addressed using various methods, such as iterative minimisation schemes or metaheuristic approaches like genetic algorithms. One of the simplest and most commonly employed methods is the *Picard iteration* [1], analogous to the iterative process in the k -means algorithm. In this case, the iteration arises from the first-order conditions for stationary points of the objective functional J (Equation (4)), which defines the FCM algorithm. Unlike the k -means method, the FCM algorithm minimises a weighted sum of errors, where the weights correspond now to a fuzzy partition of the data. This additional flexibility requires imposing a constraint on the fuzzy membership degrees, which can be incorporated into the distortion cost function J using the method of Lagrange multipliers. Specifically, Lagrange multipliers λ_k are introduced to enforce the constraint:

$$\sum_{i=1}^c w_{ik} = 1 \quad \forall k = 1, \dots, N. \quad (5)$$

The Lagrangian function becomes:

$$\bar{J}(\mathbf{X}; \mathbf{W}, \mathbf{M}, \boldsymbol{\lambda}) = \sum_{i=1}^c \sum_{k=1}^N (w_{ik})^m D_{ik\mathbf{A}}^2 + \sum_{k=1}^N \lambda_k \left[\sum_{i=1}^c w_{ik} - 1 \right]. \quad (6)$$

By setting the gradients of $\bar{J}$ with respect to $\mathbf{W}$, $\mathbf{M}$, and $\boldsymbol{\lambda}$ to zero, (similar to the procedure followed in the derivation of the iterative k -means algorithm [4]), it can be shown (see, for instance [2]) that $(\mathbf{W}, \mathbf{M}) \in M_{fc} \times \mathbb{R}^{N \times D}$ may minimise eq. (6) if and only if

$$w_{ik} = \frac{1}{\sum_{j=1}^c \left(\frac{D_{ik\mathbf{A}}}{D_{jk\mathbf{A}}} \right)^{2/(m-1)}}, \quad 1 \leq i \leq c, 1 \leq k \leq N, \quad (7)$$

and also:

$$\boldsymbol{\mu}_i = \frac{\sum_{k=1}^N (w_{ik})^m \mathbf{x}_k}{\sum_{k=1}^N (w_{ik})^m}, \quad 1 \leq i \leq c. \quad (8)$$

These conditions are both necessary and sufficient, as proven in [3], and provide an effective framework for optimising the functional $\bar{J}$ iteratively. The optimisation process alternates between Equations (7) and (8) until the change in successive iterations of either the membership matrix $\mathbf{W}$ or the centroid matrix $\mathbf{M}$ becomes sufficiently small. Furthermore, these conditions ensure that all constraints associated with a fuzzy partition matrix, including but not limited to constraint (5), are satisfied. The updated cluster centroids, derived from Equation (8), correspond to a weighted mean (centre of gravity) of the data points assigned to each cluster, where the weights are determined by the membership degrees w_{ik} . This formulation is a direct generalisation of Equation (3) with $m = 1$, which corresponds to the hard-partitioning limit. This relationship highlights why the algorithm is referred to as the “c-means” method. Similarly, the updated membership values, given by Equation (7),

generalise the hard assignments defined in Equation (2). In this way, the fuzzy c-means algorithm extends the classical k -means approach to account for fuzzy, or partial, memberships.

The FCM algorithm follows then an alternating optimisation scheme that iteratively updates the membership matrix $\mathbf{W}$ and the cluster centroids $\mathbf{M}$ as follows: We initialise choosing the number of clusters c , fuzziness parameter m , distance metric $\mathbf{A}$, and a small tolerance ε . We initialise the membership matrix $\mathbf{W}^{(0)}$. Then, we compute the updated cluster centres or centroids $\mathbf{M}^{(l)}$ using equation (8). Then the membership matrix is updated w_{ik} , using equation (7), and we continue iterating until some convergence criteria is met like:

$$\|\mathbf{W}^{(l)} - \mathbf{W}^{(l-1)}\| < \varepsilon \quad \text{or} \quad \|\mathbf{M}^{(l)} - \mathbf{M}^{(l-1)}\| < \varepsilon.$$

For instance, convergence can be checked using the maximum change in membership values with the infinity norm [1]:

$$\max_{i,k} |w_{ik}^{(l)} - w_{ik}^{(l-1)}| < \varepsilon.$$

A pseudocode describing this basic FCM algorithm is presented in Algorithm 1:

Algorithm 1 Basic Fuzzy c -Means Clustering Algorithm (FCM)

Require: Data set $\{\mathbf{x}_i\}_{i=1}^N \subset \mathbb{R}^D$, number of clusters $1 < c < N$, fuzziness parameter $m > 1$, termination tolerance $\varepsilon > 0$, and norm-inducing matrix $\mathbf{A}$. Initialise partition matrix $\mathbf{W}^{(0)}$ randomly such that $\sum_{i=1}^c w_{ik} = 1$ for all k .

repeat

Step 1: Compute cluster prototypes or centroids:

$$\boldsymbol{\mu}_j^{(l)} = \frac{\sum_{k=1}^N \left(w_{kj}^{(l-1)}\right)^m \mathbf{x}_k}{\sum_{k=1}^N \left(w_{kj}^{(l-1)}\right)^m}, \quad 1 \leq j \leq c.$$

Step 2: Compute the distances:

$$D_{ik\mathbf{A}}^2 = (\mathbf{x}_k - \boldsymbol{\mu}_i^{(l)})^\top \mathbf{A} (\mathbf{x}_k - \boldsymbol{\mu}_i^{(l)}), \quad 1 \leq i \leq c, \quad 1 \leq k \leq N.$$

Step 3: Update the partition matrix:

for $1 \leq k \leq N$

if $D_{ik\mathbf{A}} > 0$ **for all** $i = 1, 2, \dots, c$

$$w_{ik}^{(l)} = \frac{1}{\sum_{j=1}^c (D_{ik\mathbf{A}}/D_{jk\mathbf{A}})^{2/(m-1)}}.$$

else

$$w_{ik}^{(l)} = 0 \text{ if } D_{ik\mathbf{A}} > 0, \text{ and } w_{ik}^{(l)} \in [0, 1] \text{ with } \sum_{i=1}^c w_{ik}^{(l)} = 1.$$

until $\|\mathbf{W}^{(l)} - \mathbf{W}^{(l-1)}\| < \varepsilon$.

return Cluster centres $\{\boldsymbol{\mu}_i\}_{i=1}^c$ and fuzzy partition matrix $\mathbf{W} \in \mathbb{R}^{c \times N}$ (degrees of membership). =0

As happened with the basic k -means algorithm, the iterative FCM algorithm ensures convergence but only to a *local* minimum of the c -means functional. Therefore, the result may depend on the initial values of the centroids. This is a restricting and shared limitation with the k -means algorithm. On the other hand, from equation (7) it is seen that a singularity in FCM occurs when $D_{ik\mathbf{A}} = 0$ for some $\mathbf{x}_k$ and one or more $\boldsymbol{\mu}_i$. In such rare cases, zero membership is assigned to the clusters for which $D_{ik\mathbf{A}} > 0$ and the weight memberships are then distributed equally among the clusters for which $D_{ik\mathbf{A}} = 0$, with the constraint $\sum_{i=1}^k w_{ik} = 1$ (this is the else condition stated in Step 3 of Algorithm 1). We also highlight that the final output is a probabilistic assignment rather than a hard assignment as with k -means.

5 Parameter Selection for the FCM Algorithm

The performance of the fuzzy c -means (FCM) algorithm heavily depends on the proper selection of its parameters. The key parameters that need to be defined before running the algorithm are:

5.1 Number of Clusters (c)

When clustering data without any prior information about their structure, it is common to have to make an assumption about the number of clusters c in the underlying dataset. The influence of this assumption (the appropriate value of the parameter c) is key in the calculation of the algorithm, since the algorithm will search for c clusters, regardless of whether they actually exist as such or not, which is why c is the most critical parameter in the FCM algorithm. There are two main approaches to attempting to correctly predetermine such number c :

- **Using Validity Measures:** Validity measures evaluate the quality of a partition by quantifying aspects such as the separation between clusters and their compactness. When the number of clusters matches the correct number of distinguishable groups in the data, the clustering algorithm can be expected to identify them correctly. One popular validity measure for FCM is the Xie-Beni index, defined as:

$$\chi(\mathbf{X}; \mathbf{W}, \mathbf{M}) = \frac{\sum_{i=1}^c \sum_{k=1}^N (w_{ik})^m \|\mathbf{x}_k - \boldsymbol{\mu}_i\|^2}{c \cdot \min_{i \neq j} \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|^2}. \quad (9)$$

The optimal number of clusters minimises this index and can be interpreted as the ratio of the total within-group variance and the separation of the cluster centroids.

- **Iterative Merging or Insertion of Clusters:** In this approach, clustering is performed iteratively by either merging similar clusters or splitting existing clusters based on some well-defined criteria. Two possible approaches are:
 - Start with a large number of clusters and merge clusters that are close or similar.
 - Begin with a small number of clusters and add clusters in regions where membership degrees are low.

5.2 Fuzziness Parameter (m)

The weighting exponent m is another important parameter, as it influences the degree of fuzziness in the partition. For larger values of m the optimal degree of membership is more shared between clusters:

- As $m \rightarrow 1$, as it is easily deduced from equation 7, the optimal membership approaches a hard clustering partition, where $w_{ik} \in \{0, 1\}$.
- As $m \rightarrow \infty$, the optimal partition (again from equation 7) becomes completely fuzzy, with $w_{ik} = 1/c$ for all i, k , so that every points belongs to every cluster with the same probability.

A useful range of values for m lies in the interval $[1, 30]$, and the optimal value is usually determined empirically. One very typical choice is $m = 2$, balancing interpretability and flexibility.

5.3 Norm-Inducing Matrix ($\mathbf{A}$, $\|\cdot\|_A$)

The choice of the norm-inducing matrix $\mathbf{A}$ determines the shape of the clusters. Popular options include:

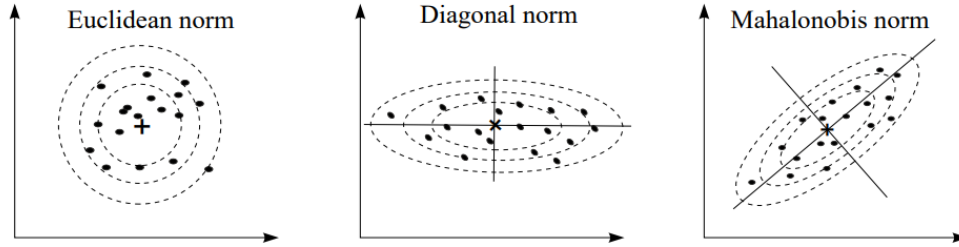


Figure 1: Different distance norms used in fuzzy clustering. Figure reproduced from [1].

- **Euclidean Norm:** $\mathbf{A} = \mathbf{I}$, leading to hyperspherical clusters.
- **Diagonal Norm:** $\mathbf{A} = \text{diag}(1/\sigma_1^2, \dots, 1/\sigma_D^2)$, where σ_i represents the standard deviation of the i -th feature in the data set. This norm accounts for differing variances across dimensions, allowing for axis-aligned hyperellipsoidal clusters.
- **Mahalanobis Norm:** $\mathbf{A} = \mathbf{R}^{-1}$, where $\mathbf{R}$ is the covariance matrix of the data:

$$\mathbf{R} = \frac{1}{N} \sum_{k=1}^N (\mathbf{x}_k - \bar{\mathbf{x}})(\mathbf{x}_k - \bar{\mathbf{x}})^T, \quad (10)$$

and $\bar{\mathbf{x}} = \frac{1}{N} \sum_{k=1}^N \mathbf{x}_k$ is the mean vector of the data. This norm generalises the clustering to arbitrary hyperellipsoidal shapes (arbitrary orientation, not necessarily aligned with the axes) by considering correlations among features.

A limitation of fixed $\mathbf{A}$ is the inability to accommodate clusters of differing shapes within a dataset. Advanced methods, such as the Gustafson-Kessel algorithm, address this by adapting the matrix $\mathbf{A}$ for each cluster, by considering local covariances.

5.4 Initial Partition of $\mathbf{W}$

The partition matrix $\mathbf{W}$ is typically initialised randomly, ensuring $\mathbf{W} \in \mathcal{M}_{fc}$. Alternatively, initial cluster centers $\boldsymbol{\mu}_i$ can be chosen randomly, and $\mathbf{W}$ computed using:

$$w_{ik} = \left(\sum_{j=1}^c (D_{ik\mathbf{A}}/D_{jk\mathbf{A}})^{2/(m-1)} \right)^{-1}. \quad (11)$$

The choice of initialisation can influence convergence to local minima, emphasising the importance of careful selection or multiple runs. As with k -means, a common strategy is to run the algorithm multiple times and choose the result with the best distortion.

6 Limitations of the Basic FCM Algorithm

The Basic Fuzzy c -Means (FCM) algorithm is a widely used clustering method due to its simplicity and ability to assign data points to multiple clusters with varying degrees of membership. However, it has several limitations that can affect its performance in certain scenarios:

1. **Sensitivity to Cluster Shape:** The standard FCM algorithm uses a fixed distance norm which constrains clusters to be hyper-ellipsoidal. This rigidity prevents the algorithm from effectively modelling data with clusters of varying shapes, such as elongated or concentric shaped clusters.
2. **Fixed Norm-Inducing Matrix:** By employing a single norm-inducing matrix across all clusters, the FCM algorithm struggles to adapt to datasets containing clusters with differing geometries. This limitation necessitates adaptive approaches to better capture the data structure.
3. **Dependence on Initialisation:** The outcome of the FCM algorithm is highly dependent on its initial parameters, including cluster centers and membership values. Poor initialisation often leads to convergence to local minima, resulting in suboptimal clustering.
4. **Constraint on Membership Values:** In the basic FCM algorithm, the membership values of a data point across all clusters must sum to 1 ($\sum_{i=1}^c \mu_{ik} = 1, \forall k$). This constraint poses a problem for noisy data points or outliers. Such points, even if far from all cluster centers, are forced to have significant membership in one or more clusters. This can distort the computation of cluster centroids and reduce the quality of the clustering.

Several extensions to the basic FCM algorithm, detailed in Section 7, have been proposed to address these limitations. These advancements build upon the basic FCM framework, providing greater flexibility and applicability to a wide range of realistic datasets.

7 Extensions of the Basic FCM Algorithm

There are several well-established extensions of the standard fuzzy c -means algorithm that we discuss now.

7.1 Adaptative distance measure algorithms

As we have seen, the shape of the clusters is primarily determined by the choice of norm-inducing matrix $\mathbf{A}$. The common choice $\mathbf{A} = \mathbf{I}$ gives the standard Euclidean norm, which in turn induces hyperspherical clusters. There are other common choices such as the diagonal norm or the Mahalanobis norm, which give hyperellipsoidal clusters, as we discussed in section 5. A common downside of clustering algorithms where the distance norm is fixed is that the objective functions is constrained to favour a particular shape for the clusters even if this is not present in the data. We illustrate this with the following example on a synthetic data set in $\mathbb{R}^2$ with two well-separated clusters of different shapes. The basic FMC algorithm was applied to this data set, with the norm-inducing matrix $\mathbf{A} = \mathbf{I}$, fuzziness parameter $m = 2$ and termination criterion $\varepsilon = 0.01$. As we can see from Figure 2, the cluster membership levels curves enforce a circular shape, despite the fact that the lower cluster is considerable elongated horizontally.

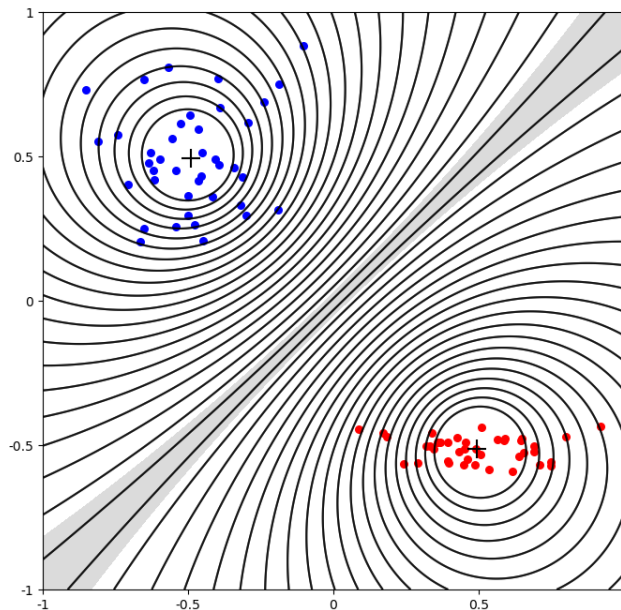


Figure 2: The fuzzy c -means algorithm with the Euclidean norm imposes spherical symmetry on the clusters, irrespective of the real shape of the data. The cluster membership level curves are shown along with the cluster mean, denoted by $+$. The shaded region indicates membership degrees close to 0.5. Plot produced using the `scikit-fuzzy` library in Python.

Unfortunately, using any other $\mathbf{A}$ would be of no help either, given that the two clusters have different shapes in this case. In general, one needs to use different matrices $\mathbf{A}_i$, but there is no prescription as to how to choose these. There is an alternative: to adapt such matrices by using estimates of the data covariance. We now present the Gustafson-Kessel algorithm, which employs such an adaptive distance norm.

7.1.1 Gustafson-Kessel Algorithm

Gustafson and Kessel [6] proposed a variation of fuzzy c -means clustering (Algorithm 1) which incorporates an adaptive distance norm to effectively capture different geometric shapes of the clusters in a given data set. In this model, each cluster has its own norm-inducing matrix $\mathbf{A}_i$, which, as before, induces the inner product norm

$$d_{ik\mathbf{A}_i}^2 = \|\mathbf{x}_k - \boldsymbol{\mu}_i\|_{\mathbf{A}_i}^2 = (\mathbf{x}_k - \boldsymbol{\mu}_i)^\top \mathbf{A}_i (\mathbf{x}_k - \boldsymbol{\mu}_i).$$

The matrices $\mathbf{A}_i$ are treated as variables in the c -means functional (Eq. 4), enabling each cluster to adjust the distance norm to the local topological structure of the data. The GK algorithm objective functional is then given by

$$J(\mathbf{X}; \mathbf{W}, \mathbf{M}, \{\mathbf{A}_i\}) = \sum_{i=1}^c \sum_{k=1}^N (w_{ik})^m d_{ik\mathbf{A}_i}.$$

To render the minimisation of J to the matrices $\mathbf{A}_i$ tractable, each $\mathbf{A}_i$ is constrained to have positive determinant

$$|\mathbf{A}_i| = \rho_i, \quad \rho_i > 0, \quad \forall i.$$

Allowing $\mathbf{A}_i$ to vary while keeping its determinant fixed is equivalent to optimising the corresponding cluster's shape with its volume unchanged. The optimal matrices are obtained with the Lagrange multiplier method. The Lagrangian function may be written as

$$\mathcal{L}(\{\mathbf{A}_i\}, \{\lambda_i\}) = \sum_{i=1}^c \sum_{k=1}^N (w_{ik})^m (\mathbf{x}_k - \boldsymbol{\mu}_i)^\top \mathbf{A}_i (\mathbf{x}_k - \boldsymbol{\mu}_i) - \sum_{i=1}^c \lambda_i (\det(\mathbf{A}_i) - \rho_i),$$

where $\{\lambda_i\}$ are Lagrange multipliers that enforce the constant determinant restriction. Setting the gradient of $\mathcal{L}$ with respect to the Lagrange multipliers to zero recovers the constraints $\det(\mathbf{A}_i) = \rho_i$, as required. For the optimal matrices $\{\mathbf{A}_i^*\}_{i=1}^c$, we note that $\nabla_{\mathbf{A}_i} (\mathbf{x}^\top \mathbf{A}_i \mathbf{x}) = \mathbf{x} \mathbf{x}^\top$ and $\nabla_{\mathbf{A}_i} \det(\mathbf{A}_i) = \det(\mathbf{A}_i) \cdot \mathbf{A}_i^{-1}$. Hence

$$\begin{aligned} \nabla_{\mathbf{A}_j} \mathcal{L} &= \sum_{k=1}^N (w_{jk})^m (\mathbf{x}_k - \boldsymbol{\mu}_j)(\mathbf{x}_k - \boldsymbol{\mu}_j)^\top - \lambda_j^* \det(\mathbf{A}_j^*) (\mathbf{A}_j^*)^{-1} \\ &:= \mathbf{F}_j - \lambda_j^* \det(\mathbf{A}_j^*) (\mathbf{A}_j^*)^{-1} = \mathbf{0} \in \mathbb{R}^{n \times n}. \end{aligned}$$

Now, the fact that $\det(\mathbf{A}_j) = \rho_j$ implies $\mathbf{F}_j = \lambda_j^* \rho_j (\mathbf{A}_j^*)^{-1}$, so

$$\lambda_j^* \mathbf{I}_n = (\rho_j)^{-1} \mathbf{F}_j \mathbf{A}_j^*,$$

where $\mathbf{I}_n$ denotes the $n \times n$ identity matrix. Taking determinants yields

$$(\lambda_j)^n = (\rho_j)^{-n} \rho_j \det \mathbf{F}_j,$$

thus

$$\lambda_j^* = (\rho_j)^{-1} (\rho_j \det \mathbf{F}_j)^{1/n}.$$

Substituting this into the formula for $\mathbf{F}_j$ gives

$$\mathbf{F}_j = (\rho_j \det \mathbf{F}_j)^{1/n} (\mathbf{A}_j^*)^{-1}$$

Rearranging this last equation, we obtain the following closed form solution for the optimal norm-inducing matrices:

$$\mathbf{A}_i^* = (\rho_i \det \mathbf{F}_i)^{1/n} \mathbf{F}_i^{-1},$$

where $\mathbf{F}_i$ is called the *fuzzy covariance matrix* of the i th cluster and is given by

$$\mathbf{F}_i = \frac{\sum_{k=1}^N (w_{ik})^m (\mathbf{x}_k - \boldsymbol{\mu}_i)(\mathbf{x}_k - \boldsymbol{\mu}_i)^\top}{\sum_{k=1}^N (w_{ik})^m}.$$

This gives a generalised Mahalanobis norm distance, where the covariance is weighted with the associated membership degrees in $\mathbf{W}$. The Gustafson-Kessel (GK) Algorithm is now presented. It is more computationally expensive than the standard FCM algorithm, since the inverse and determinant of the cluster covariance matrix need to be calculated at each iteration.

Algorithm 2 Gustafson-Kessel Algorithm

Require: Data set $\{\mathbf{x}_i\}_{i=1}^N \subset \mathbb{R}^n$, number of clusters $1 < c < N$, fuzziness parameter $m > 1$, termination tolerance $\varepsilon > 0$ and cluster volumes ρ_i . Initialise partition matrix randomly such that $\mathbf{W}^{(0)} \in M_{fc}$.

repeat

Step 1: Compute cluster prototypes:

$$\boldsymbol{\mu}_i^{(l)} = \frac{\sum_{k=1}^N \left(w_{ik}^{(l-1)}\right)^m \mathbf{x}_k}{\sum_{k=1}^N \left(w_{ik}^{(l-1)}\right)^m}, \quad 1 \leq i \leq c.$$

Step 2: Compute the cluster covariance matrices:

$$\mathbf{F}_i = \frac{\sum_{k=1}^N (w_{ik})^m (\mathbf{x}_k - \boldsymbol{\mu}_i^{(l)})(\mathbf{x}_k - \boldsymbol{\mu}_i^{(l)})^\top}{\sum_{k=1}^N (w_{ik})^m}.$$

Step 3: Compute the distances:

$$d_{ik\mathbf{A}_i}^2 = (\mathbf{x}_k - \boldsymbol{\mu}_i^{(l)})^\top \left[(\rho_j \det \mathbf{F}_j)^{1/n} \mathbf{F}_i^{-1} \right] (\mathbf{x}_k - \boldsymbol{\mu}_i^{(l)}), \quad 1 \leq i \leq c, \quad 1 \leq k \leq N.$$

Step 4: Update the partition matrix:

for $1 \leq k \leq N$

if $d_{ik\mathbf{A}_i} > 0$ **for all** $i = 1, 2, \dots, c$

$$w_{ik}^{(l)} = \frac{1}{\sum_{j=1}^c (d_{ik\mathbf{A}_i} / d_{jk\mathbf{A}_i})^{2/(m-1)}}.$$

else

$$w_{ik}^{(l)} = 0 \text{ if } d_{ik\mathbf{A}_i} > 0, \text{ and } w_{ik}^{(l)} \in [0, 1] \text{ with } \sum_{i=1}^c w_{ik}^{(l)} = 1.$$

until $\|\mathbf{W}^{(l)} - \mathbf{W}^{(l-1)}\| < \varepsilon$.

return Cluster centres $\{\boldsymbol{\mu}_i\}_{i=1}^c$ and partition matrix $\mathbf{W}$. =0

7.2 Application of the GK algorithm to synthetic data

The GK algorithm was applied to the same synthetic data set as before with the same parameter values. A circular cluster and an ellipsoid are obtained. See Figure 3.

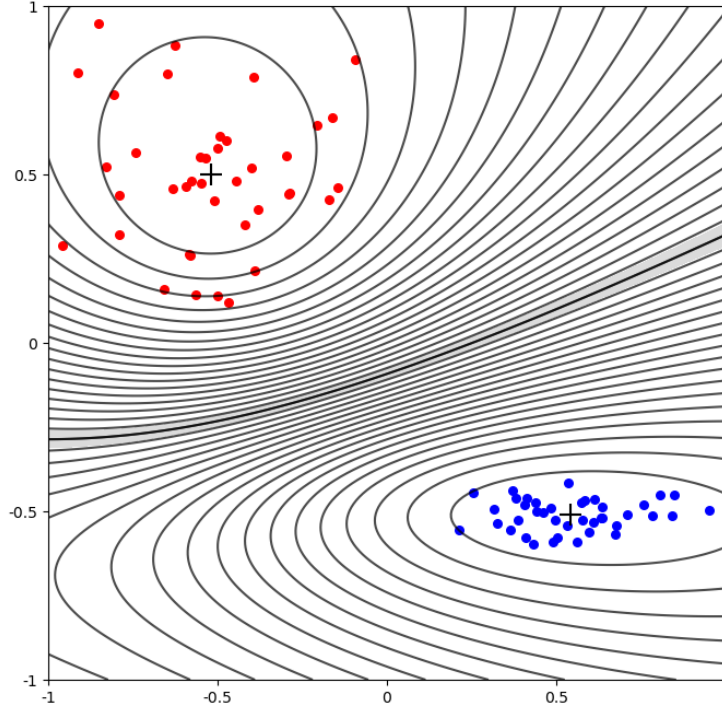


Figure 3: The Gustafson-Kessel algorithm is able to detect clusters with various shapes and orientations. Level curves of cluster membership are shown. The shaded region indicates membership degrees close to 0.5. Plot produced implementing the GK algorithm manually in Python.

7.3 Possibilistic c-Means Algorithm

The Possibilistic C-Means (PCM) algorithm is a clustering method developed as an extension of the Fuzzy C-Means (FCM) approach to overcome its limitations, particularly sensitivity to noise and outliers, developed by [7]. Unlike FCM, PCM introduces a *possibilistic partition* that allows clusters to model degrees of membership independently. Traditional fuzzy clustering assigns memberships w_{ij} that sum to 1 for all data points, which can lead to issues when clusters are not well-defined. Instead, PCM replaces the constraint on fuzzy memberships with a possibilistic framework that allows more freedom by decoupling cluster memberships. PCM interpret then these values as degrees of compatibility with clusters, relaxing the probabilistic constraint.

The membership values $w_i(\mathbf{x}_k) = w_{ik} \in [0, 1]$ now satisfy the following constraints:

$$0 \leq w_i(x) \leq 1, \forall i = 1, \dots, c, \quad 0 < \sum_{k=1}^N w_i(x_k) < N, \quad \max_i w_i(x_k) > 0, \forall k.$$

The algorithm minimises a modified least squared error function (distortion) to generate the clustering results. The PCM objective function introduces a penalty term to avoid trivial solutions:

$$J_m(\mathbf{W}, \mathbf{M}, \boldsymbol{\eta}) = \sum_{k=1}^N \sum_{i=1}^c (w_{ik})^m d_{ik\mathbf{A}}^2 + \sum_{i=1}^c \eta_i \sum_{k=1}^N (1 - w_{ik})^m,$$

where w_{ik} is the membership degree of $\mathbf{x}_k$ to cluster i , $d_{ik\mathbf{A}}^2 = \|\mathbf{x}_k - \boldsymbol{\mu}_i\|_{\mathbf{A}}^2 = (\mathbf{x}_k - \boldsymbol{\mu}_i)^\top \mathbf{A} (\mathbf{x}_k - \boldsymbol{\mu}_i)$ is the induced $\mathbf{A}$ -distance between $\mathbf{x}_k$ and $\boldsymbol{\mu}_i$ (as discussed previously), $m > 1$ is the fuzzifier parameter, and η_i , for $i = 1 \dots, c$ controls the cluster's size, shape and spreading. This cluster size parameters η_i can be calculated as [7]:

$$\eta_i = \frac{\sum_{j=1}^n (w_{ij})^m d_{ij\mathbf{A}}^2}{\sum_{j=1}^n (w_{ij})^m}, \quad i = 1, \dots, c.$$

The most general PCM algorithm iterates through the following steps:

1. **Initialise:** Set the number of clusters c , the initial cluster centroids $\boldsymbol{\mu}_j$ and initialise the membership degrees w_{ij} (now they have to be possibilistic restriction, relaxing the probability restriction).
2. **Update Memberships:**

$$w_{ij} = \frac{1}{1 + \left(\frac{d_{ij\mathbf{A}}^2}{\eta_j} \right)^{\frac{1}{m-1}}}, \quad i = 1 \dots c, \quad j = 1 \dots N$$

where $d_{ij\mathbf{A}}$ is the distance from point $\mathbf{x}_i$ to the centroid $\boldsymbol{\mu}_j$.

3. **Update Cluster Centroids:**

$$\boldsymbol{\mu}_j = \frac{\sum_{i=1}^n (w_{ij})^m \mathbf{x}_i}{\sum_{i=1}^n (w_{ij})^m}, \quad j = 1, \dots, c.$$

4. **Update cluster spread parameters η_j :**

$$\eta_j = \frac{\sum_{i=1}^n (w_{ij})^m d_{ij\mathbf{A}}^2}{\sum_{i=1}^n (w_{ij})^m}, \quad j = 1, \dots, c.$$

5. **Repeat:** Steps 2–4 until convergence, typically determined by a change threshold ε .

PCM effectively handles noisy data and outliers by decoupling memberships between clusters. It also removes the constraint that memberships across clusters sum to one, providing more flexibility. However, PCM requires careful initialisation of η_i values to ensure appropriate clustering performance, and it could converge to trivial solutions if not properly initialised.

7.4 Other Extensions of the Basic FCM Algorithm

In addition to the extensions discussed earlier, there are other noteworthy modifications to the Basic FCM algorithm that, while not explored in detail in this document, they deserve to be briefly commented on. These “enhanced approaches” are able to address more general problems and improve the adaptability of the algorithm to various data scenarios:

Algorithms Based on Hyperplanar or Functional Prototypes: These extensions replace the traditional point-based cluster prototypes with hyperplanes, curves, or other functional forms, making them better suited for data with linear or nonlinear subspace structures. Examples include:

- **Fuzzy C-Varieties:** Designed for clusters that align along hyperplanes or affine subspaces, this method minimises the within-cluster variance around hyperplanar prototypes rather than point prototypes.
- **Fuzzy Regression Models:** Here, the main ideas of the basic FCM algorithm have been carried over and applied to solve regression problems.

Enhanced Initialisation Techniques: The success of FCM heavily depends on the initial placement of cluster prototypes and membership values. A bad initialisation may result in the algorithm not converging to the global minimum, getting stuck in a local minimum that is potentially far from the optimal solution. To address this, various advanced initialisation techniques have been developed:

- **Hierarchical Pre-Clustering:** This approach uses hierarchical clustering to identify initial cluster centers, leveraging its ability to capture the overall data structure before applying FCM.
- **Density-Based Initialisation:** By analyzing regions of high data density, this method ensures initial cluster centers are placed in meaningful locations, reducing convergence time and improving accuracy.
- **Multiple Initialisations:** Running FCM multiple times with different initialisations and selecting the best result based on a cluster validity index (e.g., Xie-Beni) can further enhance the clustering outcome.

Hybrid and Specialised Algorithms: This is usually the most fruitful approach, as we have also discussed in class. Hybrid approaches combine FCM with other clustering techniques to exploit complementary strengths, addressing, for example, problems of non-spherical clusters or high-dimensional data:

- **Spectral Clustering Integration:** Combining FCM with spectral clustering leverages graph-based representations of data to uncover complex, non-linear relationships.
- **Kernel-Based FCM:** By embedding data into a high-dimensional feature space using kernel functions, this extension enables FCM to identify non-linear clusters effectively.

- **Fuzzy C-Medoids:** The main difference between FCM and Fuzzy C-Medoids (FCMdd) lies only in the **M**-formation mechanism. FCMdd selects some of the existing true objects as cluster medoids, while FCM considers some virtual objects, which are weighted average values of objects, as cluster centres. This subtle difference makes these two algorithms have different performance characteristics: FCMdd is more resistant to noise than FCM and can more easily generate clustering results with high accuracy because noisy objects will more easily affect FCM centroids.

8 Application to Data

8.1 Image Segmentation with Fuzzy Clustering

Fuzzy clustering, particularly the Fuzzy C-Means (FCM) algorithm, has proven to be an effective tool in image segmentation, allowing for the soft partitioning of image pixels based on their intensity values. This method is particularly beneficial when dealing with images that exhibit gradual transitions between regions, as opposed to sharp boundaries.

In this study, we applied the FCM algorithm to segment the image shown in Figure 4. The Python implementation of this process is detailed in the Appendix (Listing 2). The segmentation procedure follows these steps:

1. **Image Preprocessing:** The input image was converted to grayscale, reducing its complexity by working with intensity values.
2. **Feature Extraction:** The grayscale pixel values were flattened into a one-dimensional array, preparing the data for clustering.
3. **Clustering:** Using the FCM algorithm, the pixels were grouped into $c = 3$ clusters, each representing a region in the segmented image. The fuzziness parameter m was set to 2, promoting flexibility in cluster membership.
4. **Image Reconstruction:** Cluster memberships were mapped back to their original pixel positions to reconstruct the segmented image.

The segmented image, shown on the right in Figure 4, has been obtained using this method. Each region corresponds to one of the three clusters, where pixel memberships are determined by their similarity to the cluster centroids.

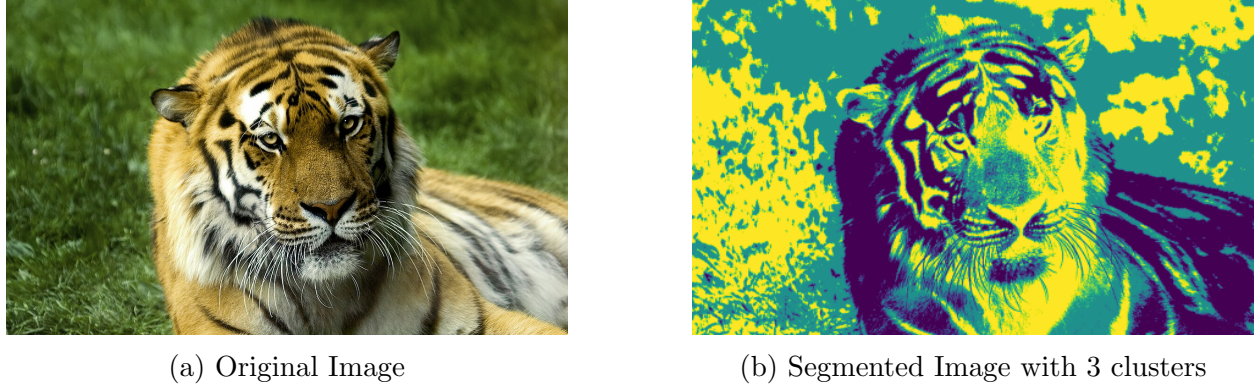


Figure 4: Application of FCM to image segmentation. The left panel shows the original image, while the right panel presents the segmented version, with $c = 3$ clusters and $m = 2$.

The probabilistic framework of fuzzy clustering makes it suitable for applications such as image segmentation, where gradual transitions are common. By assigning degrees of membership, it allows for better handling of ambiguous pixels, ensuring robust segmentation even in the presence of noise or complex boundaries. We implement the FCM algorithm on other images to illustrate several results, suggesting areas where it can find potential applications.

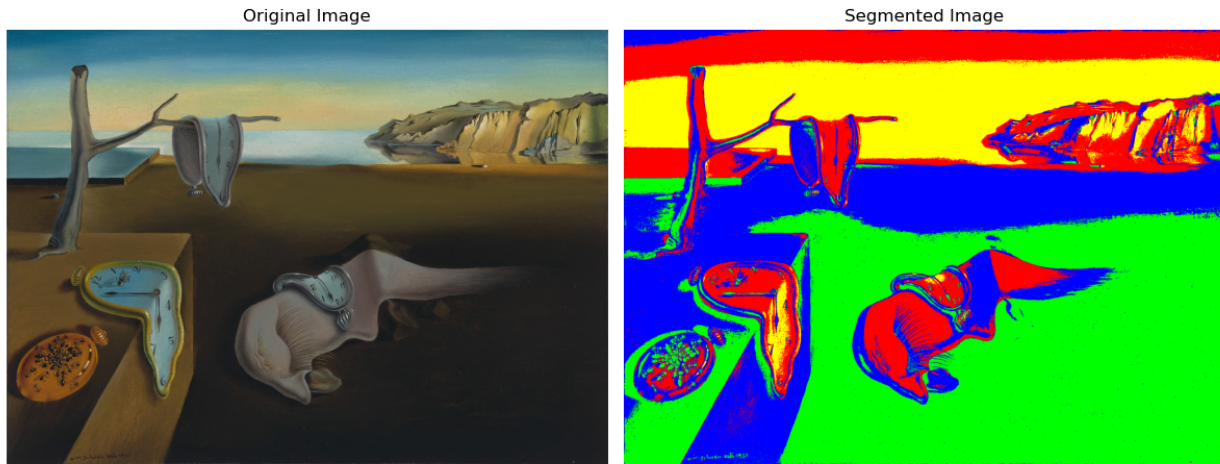


Figure 5: The fuzzy c -means algorithm applied to *The Persistence of Memory* by Salvador Dalí. Fuzziness parameter was set to $m = 2$, tolerance $\varepsilon = 0.01$ with $c = 4$ clusters. The algorithm is able to recognise isolated figures and patterns.

Additionally, fuzzy clustering is commonly used in medical applications as in medical imaging. We apply the c -means algorithm on a MRI image of a brain to illustrate this is area of potential applications. This method may be used, for instance, to identify gray matter, white matter and cerebrospinal fluid, or to detect tumors or abscesses.

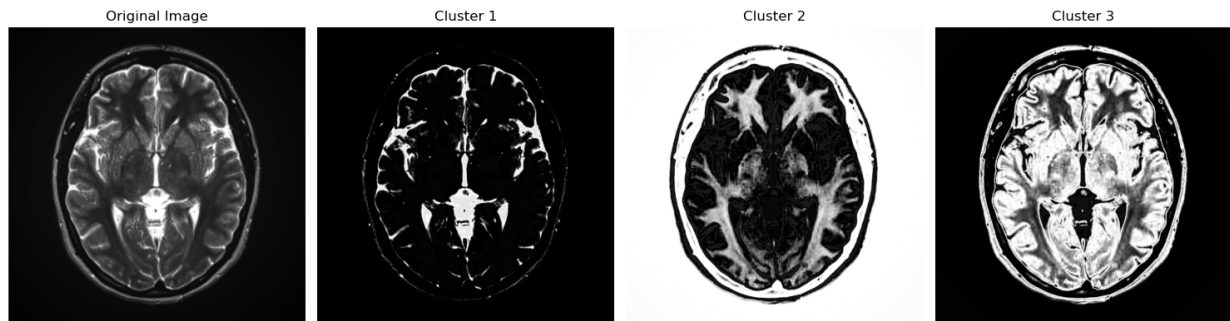


Figure 6: c -means clustering applied to a magnetic resonance imaging (MRI) scan of a brain. Shown are the original image (left-most image) along with 3 clusters. A tolerance of $\varepsilon = 0.01$ was chosen, with fuzziness parameter $m = 2$.

9 Conclusions

Fuzzy clustering offers a flexible and powerful alternative to traditional hard clustering methods, such as K-means. By allowing data points to belong to multiple clusters with varying degrees of membership, fuzzy clustering effectively captures the inherent uncertainty and overlap present in many real-world datasets. This makes it particularly well-suited for applications like image segmentation, bioinformatics and pattern recognition, where clear-cut boundaries are uncommon, and smoother transitions are desired.

The Fuzzy C-Means (FCM) algorithm, one of the most basic fundamental algorithms of fuzzy clustering, excels in handling noisy data and provides a more realistic representation of cluster membership. Additionally, its ability to adapt to various clustering scenarios through parameter tuning, such as the fuzziness parameter m , or the induced distance $\mathbf{A}$, adds to its versatility.

However, it shall be reminded that fuzzy clustering is not without its limitations. The performance of FCM heavily depends on the initial cluster centers and may converge to local minima, and still has some sensitivity to outliers. Moreover, the number of clusters (c) must be predetermined, which may not be straightforward. FCM also assumes spherical or ellipsoidal clusters, making it less effective for datasets with complex or irregular geometries. Furthermore, its computational cost is higher than that of hard clustering methods due to the iterative updates of membership probabilities and centroids. Despite these challenges, nice extensions of FCM, such as the Gustafson-Kessel algorithm and possibilistic clustering, address many of these limitations by adapting cluster shapes (by adapting the norm to the underlying distribution of the data) or improving robustness to outliers. All in all, fuzzy clustering provides a flexible and powerful alternative to hard clustering, due to its ability to handle noise, and effectiveness with overlapping clusters and we have seen that fuzzy c -means serves as the foundation for many advanced fuzzy clustering techniques .

A Python codes

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from skfuzzy.cluster import cmeans
4 from matplotlib.ticker import FormatStrFormatter
5 from scipy.spatial.distance import cdist
6
7 # Generate synthetic dataset in R^2
8 np.random.seed(19)
9
10 # Parameters for the clusters
11 mean_upper = [-0.5, 0.5]
12 std_upper = [0.2, 0.2]
13 mean_lower = [0.5, -0.5]
14 std_lower = [0.2, 0.05]
15
16 # Generate random samples for each cluster
17 num_samples = 40
18 upper_cluster = np.random.normal(mean_upper, std_upper, (num_samples, 2))
19 lower_cluster = np.random.normal(mean_lower, std_lower, (num_samples, 2))
20
21 # Combine the clusters into one dataset
22 data = np.vstack((upper_cluster, lower_cluster))
23 data = data.T # Transpose to match scikit-fuzzy's input format
24
25 # Fuzzy C-Means (FCM) parameters
26 num_clusters = 2
27 m = 2 # fuzziness parameter
28 epsilon = 0.01 # convergence threshold
29 max_iter = 100 # maximum number of iterations
30
31 # Apply Fuzzy C-Means algorithm
32 cntr, u, u0, d, jm, p, fpc = cmeans(data, c=num_clusters, m=m, error=epsilon,
    maxiter=max_iter, init=None)
33
34 # Cluster membership values
35 cluster_membership = np.argmax(u, axis=0)
36
37 # Generate a grid of points for membership level curves
38 x = np.linspace(-1, 1, 200)
39 y = np.linspace(-1, 1, 200)
40 X, Y = np.meshgrid(x, y)
41 grid = np.c_[X.ravel(), Y.ravel()]
42
43 # Calculate distances to cluster centers and membership values manually
44 distances = cdist(grid, cntr) # Compute distances to each cluster center
45 inverse_distances = 1.0 / np.fmax(distances, np.finfo(np.float64).eps) #
    Avoid divide by zero
46 u_grid = (inverse_distances.T / np.sum(inverse_distances, axis=1)).T #
    Normalise memberships
47
48 # Plot the results

```

```

49 plt.figure(figsize=(8, 8))
50
51 # Scatter plot for the clusters
52 colors = ['r', 'b']
53 for i in range(num_clusters):
54     plt.scatter(data[0, cluster_membership == i], data[1, cluster_membership
55                 == i], c=colors[i], s=30)
56
57 # Mark cluster centers
58 plt.scatter(cntr[:, 0], cntr[:, 1], marker='+', s=200, c='black')
59
60 # Plot membership level curves
61 for i in range(num_clusters):
62     Z = u_grid[:, i].reshape(X.shape) # Reshape each cluster's membership to
63     (200, 200)
64     plt.contour(X, Y, Z, levels=np.linspace(0.1, 0.9, 40), colors='black',
65                alpha=0.7)
66     # Add shaded region for membership around 0.5
67     plt.contourf(X, Y, Z, levels=[0.48, 0.52], colors=['gray'], alpha=0.1)
68
69 # Axis formatting
70 plt.xlim([-1, 1])
71 plt.ylim([-1, 1])
72 plt.xticks([-1, -0.5, 0, 0.5, 1]) # Set the x-axis ticks
73 plt.yticks([-1, -0.5, 0, 0.5, 1]) # Set the y-axis ticks
74 plt.gca().xaxis.set_major_formatter(FormatStrFormatter('%g'))
75 plt.gca().yaxis.set_major_formatter(FormatStrFormatter('%g'))
76 plt.show()

```

Listing 1: Python code used to produce Figure 2.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from numpy.linalg import det, inv
4 from matplotlib.ticker import FormatStrFormatter
5
6 # Generate synthetic dataset in R^2
7 np.random.seed(7)
8
9 # Parameters for the clusters
10 mean_upper = [-0.5, 0.5]
11 std_upper = [0.2, 0.2]
12 mean_lower = [0.5, -0.5]
13 std_lower = [0.2, 0.05]
14
15 # Generate random samples for each cluster
16 N = 40 # Number of data points per cluster
17 upper_cluster = np.vstack([
18     np.random.normal(mean_upper[0], std_upper[0], N),
19     np.random.normal(mean_upper[1], std_upper[1], N)
20 ])
21
22 lower_cluster = np.vstack([
23     np.random.normal(mean_lower[0], std_lower[0], N),

```

```

24     np.random.normal(mean_lower[1], std_lower[1], N)
25 ])
26
27 # Combine the clusters into dataset X
28 X = np.hstack((upper_cluster, lower_cluster)) # Shape: (2, 2N)
29
30 # Gustafson–Kessel algorithm parameters
31 c = 2 # Number of clusters
32 m = 2 # Fuzziness parameter
33 epsilon = 0.01 # Convergence threshold
34 max_iter = 100 # Maximum number of iterations
35
36 # Initialize membership matrix randomly
37 W = np.random.rand(c, X.shape[1])
38 W = W / np.sum(W, axis=0)
39
40 # Iterate to perform GK clustering
41 for iteration in range(max_iter):
42     # Step 1: Calculate cluster prototypes/centers
43     Wm = W * m
44     cluster_centers = (X @ Wm.T) / Wm.sum(axis=1, keepdims=True).T # Cluster
45     # centers shape: (2, c)
46
47     # Step 2: Compute covariance matrices
48     cov_matrices = []
49     for j in range(c):
50         diff = X - cluster_centers[:, j].reshape(-1, 1) # Shape: (2, N)
51         weighted_diff = Wm[j] * diff
52         cov_matrix = (weighted_diff @ diff.T) / np.sum(Wm[j])
53         cov_matrices.append(cov_matrix)
54
55     # Step 3: Update membership matrix
56     W_new = np.zeros_like(W)
57     for j in range(c):
58         for i in range(X.shape[1]):
59             diff = X[:, i] - cluster_centers[:, j]
60             distance = diff.T @ inv(cov_matrices[j]) @ diff
61             norm_term = det(cov_matrices[j]) ** (1 / 2)
62             W_new[j, i] = (norm_term * distance) ** (-1 / (m - 1))
63     W_new /= np.sum(W_new, axis=0) # Normalize columns
64
65     # Check for convergence
66     if np.linalg.norm(W_new - W) < epsilon:
67         print(f"Converged after {iteration} iterations.")
68         break
69     W = W_new
70
71 # Final cluster assignments
72 cluster_membership = np.argmax(W, axis=0)
73
74 # Generate a grid of points for membership level curves
75 x = np.linspace(-1, 1, 200)
76 y = np.linspace(-1, 1, 200)
77 X_grid, Y_grid = np.meshgrid(x, y)

```



```

77 grid = np.vstack((X_grid.ravel(), Y_grid.ravel())) # Shape: (2, 40000)
78
79 # Compute membership values for the grid
80 grid_memberships = np.zeros((c, grid.shape[1]))
81 for j in range(c):
82     for i, point in enumerate(grid.T):
83         diff = point - cluster_centers[:, j]
84         distance = diff.T @ inv(cov_matrices[j]) @ diff
85         norm_term = det(cov_matrices[j]) ** (1 / 2)
86         grid_memberships[j, i] = (norm_term * distance) ** (-1 / (m - 1))
87 grid_memberships /= np.sum(grid_memberships, axis=0, keepdims=True)
88
89 # Plot the results
90 plt.figure(figsize=(8, 8))
91
92 # Scatter plot for the clusters
93 colors = ['r', 'b']
94 for i in range(c):
95     indices = np.where(cluster_membership == i)
96     plt.scatter(X[0, indices], X[1, indices], c=colors[i], s=30)
97
98 # Mark cluster centers
99 plt.scatter(cluster_centers[0, :], cluster_centers[1, :], marker='+', s=200, c=
    ='black')
100
101 # Plot membership level curves
102 for j in range(c):
103     Z = grid_memberships[j].reshape(X_grid.shape)
104     plt.contour(X_grid, Y_grid, Z, levels=np.linspace(0, 0.5, 20), colors='
    black', alpha=0.7)
105     plt.contourf(X_grid, Y_grid, Z, levels=[0.48, 0.52], colors=['gray'],
    alpha=0.15)
106
107 # Axis formatting
108 plt.xlim([-1, 1])
109 plt.ylim([-1, 1])
110 plt.xticks([-1, -0.5, 0, 0.5, 1])
111 plt.yticks([-1, -0.5, 0, 0.5, 1])
112 plt.gca().xaxis.set_major_formatter(FormatStrFormatter('%g'))
113 plt.gca().yaxis.set_major_formatter(FormatStrFormatter('%g'))
114 plt.show()

```

Listing 2: Python code used to produce Figure 3.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from skimage import io, color
4 from skfuzzy.cluster import cmeans
5
6 # Load the image in color and grayscale
7 color_image = io.imread('imagefcm.jpg') # Original color image
8 gray_image = color.rgb2gray(color_image) # Grayscale image for clustering
9 pixels = gray_image.flatten()
10

```

```

11 # Configure the FCM algorithm
12 num_clusters = 3
13 fuzziness = 2
14 max_iter = 100
15 error = 0.005
16
17 # Apply FCM
18 cntr, u, -, -, -, -, - = cmeans(pixels.reshape(1, -1), c=num_clusters, m=
    fuzziness, error=error, maxiter=max_iter, init=None)
19
20 # Reconstruct the segmented image
21 cluster_membership = np.argmax(u, axis=0)
22 segmented_image = cluster_membership.reshape(gray_image.shape)
23
24 #Comparison of the color original image and segmented image
25 fig, ax = plt.subplots(1, 2, figsize=(12, 6))
26
27 # Original color image
28 ax[0].imshow(color_image)
29 ax[0].set_title('Original Image (Color)')
30 ax[0].axis('off') # Hide axes
31
32 # Segmented image
33 ax[1].imshow(segmented_image, cmap='viridis')
34 ax[1].set_title('Segmented Image')
35 ax[1].axis('off') # Hide axes
36
37 plt.tight_layout()
38 plt.show()

```

Listing 3: Python code used to produce Figure 4.

References

- [1] Robert Babuška. *Fuzzy modeling for control*. Vol. 12. Springer Science & Business Media, 2012.
- [2] James C Bezdek. *Pattern recognition with fuzzy objective function algorithms*. Springer Science & Business Media, 2013.
- [3] James C Bezdek, Robert Ehrlich, and William Full. “FCM: The fuzzy c-means clustering algorithm”. In: *Computers & geosciences* 10.2-3 (1984), pp. 191–203.
- [4] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*. Vol. 4. 4. Springer, 2006.
- [5] Joseph C Dunn. “Well-separated clusters and optimal fuzzy partitions”. In: *Journal of cybernetics* 4.1 (1974), pp. 95–104.
- [6] Donald E. Gustafson and William C. Kessel. “Fuzzy clustering with a fuzzy covariance matrix”. In: *1978 IEEE Conference on Decision and Control including the 17th Symposium on Adaptive Processes*. 1978, pp. 761–766. DOI: [10.1109/CDC.1978.268028](https://doi.org/10.1109/CDC.1978.268028).

-
- [7] Raghuram Krishnapuram and James M Keller. “A possibilistic approach to clustering”. In: *IEEE transactions on fuzzy systems* 1.2 (1993), pp. 98–110.